

计算机系统基础 — PA3

一、实验实现流程

1. 引言

在 PA2 中，我们完成了一个能够运行各类整数程序、能够读取键盘并绘制图像的完整冯诺依曼计算机系统，并且已经在 AM 上把打字小游戏运行了起来。然而，AM 仅仅是计算机的一种抽象模型，它对程序的运行时支撑能力相当有限：对于我们每天都在使用的“文件”这一概念，其本质其实是操作系统提供的服务。因此，要想运行规模更大、需要管理数据文件的程序，我们还需要进一步实现操作系统的支持。

PA3 的核心工作就是在 NEMU 之上运行一个简化的操作系统 Nanos-lite，并在此基础上学习理解异常控制流（exception control flow）这一支撑操作系统运行的关键机制。整体上来看，PA3 同样可以分为递进的几个阶段。阶段一以最简单的用户程序 dummy 为目标，先实现一个最朴素的 loader 把用户程序加载到内存，再在 NEMU 中补全 IDTR 寄存器、lidt、int、pusha、popa、iret 等与中断机制相关的指令，配合 AM 的 ASYE 模块构造陷阱帧、保存与恢复现场，最终实现从用户程序触发系统调用到陷入操作系统态，再到处理后返回这一完整的流程，并在此之上实现 SYS_none、SYS_exit、SYS_write、SYS_brk 几个系统调用，把 hello 程序运行起来。阶段二则在文件这一抽象上展开，主要实现了一个简易的文件系统，让 loader 可以按文件名进行加载，并依照同样的思想把 VGA 显存与设备输入也抽象为文件，最终使《仙剑奇侠传》这样的复杂程序也可以正常地在我们的 NEMU 上运行。

接下来将依照这两个阶段，逐步讲解其实现流程。需要说明的是，由于 Nanos-lite 运行在 AM 的基础上，而我们在 PA2 的报告中已经对于 AM 的实现有了详细的讲述，出于篇幅考虑，这里我们在调用时不再重复 AM 中已经写好的代码逻辑，仅对一些新增或调整做出讲解。

2. 阶段一：运行 dummy 与 Hello World

在阶段一这部分，我们的目标是让最基础的用户程序 dummy 在 Nanos-lite 上正确运行，并进一步地，把 hello 程序也运行起来。与 PA2 类似，dummy 本身几乎什么都不做，它只会触发一次系统调用而后立即退出。因此在具体实现上，我们只需实现“加

载程序—触发异常—保存现场—处理系统调用—恢复现场”这条异常控制流的基础链路即可。下面将依次讲解各个环节。

i. 实现 raw program loader

loader 的功能主要在于把可执行文件中的代码与数据放到正确的内存位置，然后返回程序入口的地址。在代码层面，实验框架其实已经把这件事情做出了极大地简化。`nanos-lite/Makefile` 通过 `objcopy` 把用户程序的代码与数据从 ELF 文件中抽取出来，整个 `ramdisk` 本身就是 loader 所需加载的内容；可执行文件位于 `ramdisk` 偏移 0 处，而正确的内存位置是约定好的 `0x4000000`。因此，loader 只需将 `ramdisk` 中从 0 开始的全部内容读到 `0x4000000`，并把该地址作为入口返回即可：

```
1 uintptr_t loader(_Protect *as, const char *filename) {
2     ramdisk_read(DEFAULT_ENTRY, 0, get_ramdisk_size());
3     return (uintptr_t)DEFAULT_ENTRY;
4 }
```

其中 `ramdisk_read` 与 `get_ramdisk_size` 由实验框架在 `ramdisk.c` 中提供。`main` 拿到返回的入口地址后，通过 `((void*)(void))entry()` 直接跳转执行。完成上述实现后运行 `dummy`，会看到它在 `0x4000000` 位置触发了一条未实现的 `int` 指令（其机器码为 `cd 80`，即 `int $0x80`），这说明 loader 已经成功加载并跳入 `dummy` 执行。这条 `int` 正是系统调用的入口，也是我们接下来要处理的对象。

ii. IDT: IDTR 寄存器与 `lidt` 指令

`int` 指令触发异常后，CPU 需要依据中断描述符表 IDT 跳转到约定的处理入口。为此，我们首先要在 NEMU 中为中断机制完成基础设施的搭建。在 `CPU_state` 中加入 IDTR 寄存器，它由 IDT 的首地址 `base` 与长度 `limit` 两部分构成；同时按照实验文档要求，再加入一个 CS 寄存器。虽然在 PA 中我们并不需要实现分段，但是当触发异常时，硬件需将 CS 压入栈中，而为了能在 QEMU 中顺利进行 differential testing，在代码中我们仍需保留该部分，并在 `restart()` 里把 CS 初始化为 8、EFLAGS 初始化为 `0x2`。

```
1 struct {
2     uint32_t base;      // IDT 首地址
3     uint16_t limit;     // IDT 长度
4 } idtr;
5 uint32_t cs;           // 仅为 differential testing 服务
```

随后，还需继续实现 `lidt` 指令。它的操作数是内存中一个 6 字节的描述符，其中低 2 字节为 `limit`，高 4 字节为 `base`，`lidt` 的工作就是把这两部分读入 IDTR：

```

1 make_EHelper(lidt) {
2     cpu.idtr.limit = vaddr_read(id_dest->addr, 2);
3     cpu.idtr.base  = vaddr_read(id_dest->addr + 2, 4);
4     print_asm_template1(lidt);
5 }

```

lidt 属于 0f 01 的 gp7 扩展组，其内存操作数的地址已由该组入口的译码器 gp7_E 解析到 id_dest->addr，因而在子表中，只需要以 EX(lidt) 注册执行函数即可。完成上述实现后，在 main.c 中启用宏定义 HAS_ASYE，让 Nanos-lite 在初始化时调用 init_irq，最终调用 AM 的 _asye_init 准备好 IDT 并执行 lidt。

iii. 触发异常：int 与 raise_intr

准备好 IDT 后，就可以真正触发并响应异常了。这一过程的核心在于 raise_intr 函数，它模拟了 i386 硬件响应中断的全过程，即：先依次将 EFLAGS、CS 和返回地址 EIP 压入堆栈，再用中断号索引 IDT 取出门描述符，组合得到目标地址，而后完成相应的跳转操作。

```

1 void raise_intr(uint8_t NO, vaddr_t ret_addr) {
2     rtl_push(&cpu.eflags.val);
3     t0 = cpu.cs;
4     rtl_push(&t0);
5     rtl_push(&ret_addr);
6
7     vaddr_t addr = cpu.idtr.base + NO * 8;
8     uint32_t off_lo = vaddr_read(addr, 4);
9     uint32_t off_hi = vaddr_read(addr + 4, 4);
10    vaddr_t target = (off_hi & 0xffff0000) | (off_lo & 0x0000ffff);
11
12    decoding.jump_eip = target;
13    decoding.is_jump = 1;
14 }

```

这里目标地址的拼接对应了实验文档中简化后的门描述符结构，低 4 字节的低 16 位为 OFFSET 的 15 - 0，高 4 字节的高 16 位为 OFFSET 的 31 - 16，两者合起来即可得到完整跳转目标的 32 位地址。值得注意的是，这里的压栈顺序 EFLAGS、CS、EIP 都是固定约定好的，恢复时 iret 也将以相反的顺序弹出。

在 int 指令这里，其本身并不实现中断逻辑，而是仅仅调用 raise_intr。这是因为在后续的设备中断 dev_raise_intr 中还会复用这一函数。其返回地址使用

`decoding.seq_eip`，即 `int` 指令的下一条指令地址，这与函数调用时保存返回地址的方式类似：

```

1 make_EHelper(int) {
2     raise_intr(id_dest->val, decoding.seq_eip);
3     print_asm("int %s", id_dest->str);
4 #ifdef DIFF_TEST
5     diff_test_skip_nemu();
6 #endif
7 }
```

在 `opcode_table` 中，`int` 的 `opcode` 为 `0xcd`。这里需要以 `IDEXW(I, int, 1)` 来进行注册，将操作数宽度强制设为 1；如果使用默认的 4 字节宽度，译码器会把 `int` 之后的若干字节误当作立即数读走，导致中断号与返回地址全部出问题报错。

完成上述实现后，再次运行 `dummy`，可以看到异常成功跳转到了 AM 的 `trap.S` 入口附近（低地址 `0x10xxxx` 一带），并在那里触发了一条未实现的 `pusha`（机器码 `0x60`）。这一现象表明，`int` 经由 `raise_intr` 查 IDT 跳转的整条逻辑链路已经实现，我们接下来只需根据报错逐步补全流程上的细节即可。

iv. 保存现场：pusha 与陷阱帧

跳转到入口函数后，软件层面开始真正的异常处理。由于处理过程不可避免地需要使用通用寄存器，而其中保存的正是异常触发前的现场，因此必须把它们先保存起来。`i386` 提供 `pusha` 指令把 8 个通用寄存器压栈，其压栈顺序由手册规定为 `EAX`、`ECX`、`EDX`、`EBX`、`ESP`、`EBP`、`ESI`、`EDI`，其中压入的 `ESP` 是执行 `pusha` 之前的原始值，因此需要预先进行保存：

```

1 make_EHelper(pusha) {
2     t0 = cpu.esp;           // 保存原始 ESP
3     rtl_push(&cpu.eax);
4     rtl_push(&cpu.ecx);
5     rtl_push(&cpu.edx);
6     rtl_push(&cpu.ebx);
7     rtl_push(&t0);
8     rtl_push(&cpu.ebp);
9     rtl_push(&cpu.esi);
10    rtl_push(&cpu.edi);
11    print_asm("pusha");
12 }
```

在 `trap.S` 中，`vecsys` 先压入错误码与异常号，再执行 `pushal`，随后把当前的 `%esp` 压栈并调用 `irq_handle`。这些压栈内容连同硬件保存的 `EFLAGS`、`CS`、`EIP`，共同构成了陷阱帧（trap frame）这一数据结构。对于异常处理来说，陷阱帧实际上非常类似于函数调用中的栈帧，它的功能主要在于完整记录用户进程触发异常时的现场，在将来进行恢复时便需要依赖于此。

为了让 `irq_handle` 能正确解释陷阱帧，必须重新组织 `arch.h` 中的 `_RegSet` 结构体，使其成员声明顺序与 `trap.S` 在栈上构造的陷阱帧一致。由于堆栈向低地址延伸，其最后压入的内容具有最低的地址，而结构体中最早定义的成员对应地址的最低处，因此结构体的成员顺序恰好是压栈顺序的逆序：

```
1 struct _RegSet {
2     uintptr_t edi, esi, ebp, esp, ebx, edx, ecx, eax;
3     int irq;
4     uintptr_t error_code, eip, cs, eflags;
5 };
```

其中 `edi` 至 `eax` 对应 `pushal` 压入的 8 个寄存器（逆序排列），`irq` 与 `error_code` 来自 `vecsys`，`eip`、`cs`、`eflags` 则来自 `raise_intr` 的压栈。这里也回答了实验文档中“诡异的 `pushl %esp`”这一问题：那条 `pushl %esp` 压入的栈顶地址恰好作为指针传给了 `irq_handle` 的形参，使函数能够据此访问整个陷阱帧，随后又被一条 `addl $4, %esp` 弹出，因此实际上它并不会计入 `_RegSet`。

完成 `pusha` 与结构体重组后，重新运行 `dummy`，会看到 `Nanos-lite` 报出“**Unhandled event ID = 8**”，这说明现场已被正确保存，陷阱帧也被正确解释，异常被封装成了 8 号的系统调用事件 `_EVENT_SYSCALL`，只是 `do_event` 还尚未对它进行处理。

v. 事件分发与系统调用处理

接下来我们的工作就是把这条 8 号事件真正处理掉。`irq_handle` 把异常封装为事件后，会交给在 `_asye_init` 中注册的处理函数，即 `nanos-lite/src/irq.c` 中的 `do_event`。我们需要在其中识别出 `_EVENT_SYSCALL`，并转交给 `do_syscall`：

```
1 static _RegSet* do_event(_Event e, _RegSet* r) {
2     switch (e.event) {
3         case _EVENT_SYSCALL: return do_syscall(r);
4         default: panic("Unhandled event ID = %d", e.event);
5     }
6     return NULL;
7 }
```

`do_syscall` 需要从陷阱帧中取出用户进程设置好的系统调用参数。观察 `libos` 中 `_syscall_` 的内联汇编，它将系统调用号与参数依次放入 `%eax`、`%ebx`、`%ecx`、`%edx`，并从 `%eax` 取回返回值。因此我们将 `arch.h` 中的取参宏对应到 `_RegSet` 的相应成员：

```
1 #define SYSCALL_ARG1(r) (r)->eax
2 #define SYSCALL_ARG2(r) (r)->ebx
3 #define SYSCALL_ARG3(r) (r)->ecx
4 #define SYSCALL_ARG4(r) (r)->edx
```

随后在 `do_syscall` 中以第一个参数（系统调用号）进行分发。`dummy` 会先触发一个 0 号的 `SYS_none`，约定其什么都不做、直接返回 1；随后触发 `SYS_exit`，它接收退出状态作为参数并调用 `_halt`。处理结束后，将返回值写回 `SYSCALL_ARG1(r)`（即陷阱帧中的 `%eax`），在恢复现场时它就会被 `popa` 弹回，用户进程便能从 `%eax` 中读到结果：

```
1 _RegSet* do_syscall(_RegSet *r) {
2     uintptr_t a[4];
3     a[0] = SYSCALL_ARG1(r); a[1] = SYSCALL_ARG2(r);
4     a[2] = SYSCALL_ARG3(r); a[3] = SYSCALL_ARG4(r);
5     switch (a[0]) {
6         case SYS_none:  SYSCALL_ARG1(r) = 1; break;
7         case SYS_exit:  _halt(a[1]); break;
8         default: panic("Unhandled syscall ID = %d", a[0]);
9     }
10    return NULL;
11 }
```

最后则是恢复现场的实现。处理完成后控制权一路返回到 `trap.S` 的 `asm_trap`，它根据陷阱帧用 `popa` 恢复通用寄存器（此时 `%eax` 已是系统调用返回值），弹出不再需要的错误码与异常号，最后执行 `iret`。`iret` 是 `raise_intr` 的逆操作，从栈顶依次弹出 EIP、CS、EFLAGS 并恢复，最后再跳转回 EIP：

```
1 make_EHelper(iret) {
2     rtl_pop(&t0); cpu.eip = t0;
3     rtl_pop(&t0); cpu.cs = t0;
4     rtl_pop(&cpu.eflags.val);
5     decoding.jump_eip = cpu.eip;
6     decoding.is_jump = 1;
7     print_asm("iret");
8 }
```

popa 的实现与 pusha 完全对称，按 EDI 至 EAX 的逆序弹出，其中对应原始 ESP 的那一项需要丢弃而不能写回，否则会扰乱栈指针。

实现上述全部内容后，再次运行 dummy，终端输出 `HIT GOOD TRAP`，标志着异常控制流的全部流程已经完整实现。在运行时，dummy 触发 `SYS_none` 取得返回值、正常推进，再触发 `SYS_exit` 进行退出。这套机制也是 PA3 后续所有系统调用实现的基础。

vi. 标准输出与堆区管理

实现了系统调用的全流程后，运行 hello 程序其实只需要再补全 `SYS_write` 即可。hello 通过 `write` 输出，依照 `man 2 write` 的约定，在 `do_syscall` 中识别出 `SYS_write` 后检查文件描述符 fd：若为 1 或 2 (stdout/stderr)，则用 `_putc` 将缓冲区内容逐字节输出到串口，并返回成功写入的字节数；同时还需在 libos 的 `_write` 中真正调用系统调用接口，使 `write` 经 `_write`、`_syscall_` 触发 int 进入内核。

实现后运行 hello，可以看到持续输出的 `Hello World for the N-th time` 打印在终端上。

```

Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 23:48:47, Jun  9 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 01:29:55, Jun 13 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101024, end = 0x105600, size = 17884 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
Hello World!
Hello World for the 2th time
Hello World for the 3th time
Hello World for the 4th time
Hello World for the 5th time
Hello World for the 6th time
Hello World for the 7th time
Hello World for the 8th time
Hello World for the 9th time
Hello World for the 10th time
Hello World for the 11th time
Hello World for the 12th time
Hello World for the 13th time
Hello World for the 14th time
Hello World for the 15th time
Hello World for the 16th time
Hello World for the 17th time
Hello World for the 18th time

```

图 1: hello 运行结果

进一步地，如果观察 `sys_write` 的调用情况可以发现，此时 `printf` 是逐个字符地调用 `write` 进行输出的。这其实是因为用户程序首次调用 `printf` 时会尝试用 `malloc` 申请格式化缓冲区，而 `malloc` 依赖堆区，堆区大小又需通过系统调用向操作系统申请。实验框架中 libos 的 `_sbrk` 总是返回 `-1`，导致每次申请都会失败，

printf 只好退化为逐字符输出。

为此，我们还需继续实现堆区的管理，在 libos 的 `_sbrk` 中以 `_end` 符号（链接器标记的数据段结束位置）为起点记录 program break，每次根据增量计算出新的 break，通过 `SYS_brk` 系统调用通知内核，成功则更新记录并返回旧 break：

```

1 void *_sbrk(intptr_t increment){
2     if (program_break == NULL) program_break = &_amp;_end;
3     char *old_break = program_break;
4     char *new_break = program_break + increment;
5     if (_syscall_(SYS_brk, (uintptr_t)new_break, 0, 0) == 0) {
6         program_break = new_break;
7         return old_break;
8     }
9     return (void *)-1;
10 }

```

而由于当前 Nanos-lite 是单任务操作系统，空闲内存可由用户程序自由使用，因此内核侧的 `SYS_brk` 只需总是返回 0 表示调整成功即可。实现后再通过添加的 Log 观察打印信息，可以看到 printf 已经改为将整行格式化结果通过单次 write 输出，而非逐字符输出。这也正是缓冲区与 batching 所发挥的作用。需要注意的是，调试 `_sbrk` 时不能直接使用 printf，否则会经 malloc 再次调用 `_sbrk` 从而造成死循环的递归，应该改用 sprintf 来配合 write 输出。

截止到这里，我们已经得到了一个较为完善的系统调用结构，对于一些基础的程序来说可以完整而正确地运行。然而，在实际的操作系统中，还有文件这一核心的对象，我们当前的 Nanos-lite 还无法实现文件读写等操作，或者，更加确切地说，系统甚至不知道什么是文件。在接下来的部分，我们还需继续完成文件的相关概念的构建。

3. 阶段二：文件系统与设备

在阶段一中，ramdisk 仅有一个文件，loader 直接读取偏移 0 即可。但是随着文件数量增多，用户程序无从知晓其中某个文件位于 ramdisk 的具体位置，直接暴露 ramdisk 的读写接口已经不再可行。操作系统需要在存储介质之上为用户程序提供更加高级的抽象，也就是我们所说的“文件”。阶段二便围绕这一抽象展开。

i. 实现简易文件系统

在文件系统部分，实验框架也作了大量的简化，每个文件大小固定，写入时不可超出原大小；文件的数量也是固定的，不支持创建新文件等操作，也没有文件目录等额外的索引。因此，我们可以用一张固定的文件记录表来描述各文件的名称、大小与在

ramdisk 中的偏移，并为每个文件增加一个 `open_offset` 字段来记录当前的读写位置。文件描述符则直接用文件记录表的下标表示。通过上述描述也可以看出，文件操作的实现逻辑其实都相当地直接，这里以 `fs_open` 与 `fs_read` 为例：

```

1 int fs_open(const char *pathname, int flags, int mode) {
2     for (int i = 0; i < NR_FILES; i++)
3         if (strcmp(file_table[i].name, pathname) == 0) {
4             file_table[i].open_offset = 0;
5             return i;
6         }
7     panic("fs_open: file %s not found", pathname);
8     return -1;
9 }
10
11 ssize_t fs_read(int fd, void *buf, size_t len) {
12     Finfo *f = &file_table[fd];
13     if (f->open_offset + len > f->size) len = f->size - f->open_offset;
14     ramdisk_read(buf, f->disk_offset + f->open_offset, len);
15     f->open_offset += len;
16     return len;
17 }

```

由于文件不会动态创建，`fs_open` 找不到文件的情况属于异常，需要用 `assertion` 来终止；同时，由于文件大小固定，`fs_read`、`fs_write`、`fs_lseek` 也都必须做边界裁剪，使其读写不越过文件末尾；`fs_lseek` 依据 `SEEK_SET/CUR/END` 三种基准计算新偏移；`fs_close` 因不维护打开状态而直接返回 0。此外还提供辅助函数 `fs_filesz` 返回文件大小，供 loader 使用。完成后，相应地还需在 `do_syscall` 中接出 `SYS_open/read/write/close/lseek`，并在 `libos` 中让 `_open` 等函数真正发起系统调用。

随后，还需对 `nanos-lite/Makefile` 进行修改，将 ramdisk 的生成规则由单文件 `objcopy` 改为基于 `navy-apps/fsimg` 的多文件方式。执行 `make update` 时，会编译 `Navy-apps` 下的所有程序，将 `fsimg` 目录的全部内容整合为 ramdisk 镜像，并自动生成文件记录表 `files.h`。这样一来，loader 也可依据此改为按文件名加载，先用 `fs_open` 打开，再用 `fs_filesz` 取得大小，最后用 `fs_read` 整体读入入口地址。此后再需更换用户程序时只要修改传入 loader 的文件名即可：

```

1 uintptr_t loader(_Protect *as, const char *filename) {
2     int fd = fs_open(filename, 0, 0);
3     size_t size = fs_filesz(fd);
4     fs_read(fd, DEFAULT_ENTRY, size);

```

```

5     fs_close(fd);
6     return (uintptr_t)DEFAULT_ENTRY;
7 }

```

运行测试程序 `/bin/text`，它会进行一系列文件读写与定位操作，最终输出 `PASS!!!` 并以 `HIT GOOD TRAP` 退出，文件系统的读、写、定位与边界保护等功能均工作正常。

```

Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 23:48:47, Jun  9 2026
For help, type "help"
(nemu) c
[src/main.c,19,main] 'Hello World!' from Nanos-lite
[src/main.c,20,main] Build time: 03:21:37, Jun 14 2026
[src/ramdisk.c,26,init_ramdisk] ramdisk info: start = 0x101fa0, end = 0x1d4c5cd, size = 29664813 bytes
[src/main.c,27,main] Initializing interrupt/exception handler...
PASS!!!
nemu: HIT GOOD TRAP at eip = 0x00100032
(nemu) █

```

图 2: test 运行结果

ii. VGA 显存与设备输入

对于我们所熟知的文件，其本质其实是字节的序列。在计算机系统中，字节序列的概念可以有很多延伸，显存、键盘输入、时钟等其实也可以用同一概念来解构。依照“一切皆文件”的思想，我们把 IOE 也抽象为文件，从而以统一的接口来屏蔽设备间的差异。

对输出设备而言，串口已被抽象为 `stdout/stderr`，剩下的是 VGA。Nanos-lite 与 Navy-apps 约定，将显存抽象为文件 `/dev/fb`，屏幕大小信息则通过 `/proc/dispinfo` 提供。这里需要注意的是，上述两者都是特殊文件，在文件记录表中应该有它们的名字，但其实体并不在 `ramdisk` 中，因此必须在 `fs_read`、`fs_write` 中对它们进行重定向。具体实现上，先在 `init_fs` 中使用 IOE 提供的屏幕尺寸来初始化 `/dev/fb` 的大小（宽 × 高 × 4 字节），而后实现 `fb_write` 把字节偏移换算为屏幕坐标再调用 IOE 的 `_draw_rect`，最后在 `init_device` 中按约定格式把屏幕尺寸写入 `dispinfo` 字符串，并实现 `dispinfo_read` 以供读取：

```

1 void fb_write(const void *buf, off_t offset, size_t len) {
2     int pixel_offset = offset / 4, n = len / 4;
3     int x = pixel_offset % _screen.width;
4     int y = pixel_offset / _screen.width;
5     _draw_rect((const uint32_t *)buf, x, y, n, 1);
6 }

```

对输入设备而言，键盘与时钟的输入被封装成以换行符结尾的文本事件，抽象为文件 `/dev/events`。其中按键事件采用 `kd RETURN` 或 `ku A` 的格式，时钟事件则采用 `t 1234` 的格式。由于时钟事件可随时读取，因此需要优先处理按键事件，仅在没有按键时才返回时钟事件，否则用户程序将永远无法读到按键的输入。`events_read` 借助 IOE 的 `_read_key` 与 `_uptime` 实现，其中 `_read_key` 的返回值以高位标志按下/松开的状态为区分：

```

1 size_t events_read(void *buf, size_t len) {
2     int key = _read_key();
3     bool down = false;
4     if (key & 0x8000) { key ^= 0x8000; down = true; }
5     if (key != _KEY_NONE)
6         sprintf(buf, "%s %s\n", down ? "kd" : "ku", keyname[key]);
7     else
8         sprintf(buf, "t %d\n", (int)_uptime());
9     return strlen(buf);
10 }

```

分别让 Nanos-lite 加载 `/bin/bmptest` 与 `/bin/events` 进行验证：前者在 VGA 窗口中央正确显示出 ProjectN 的 Logo；后者持续输出时间事件，敲击键盘时则输出对应的按键事件，键名与按下/松开的状态均正确。

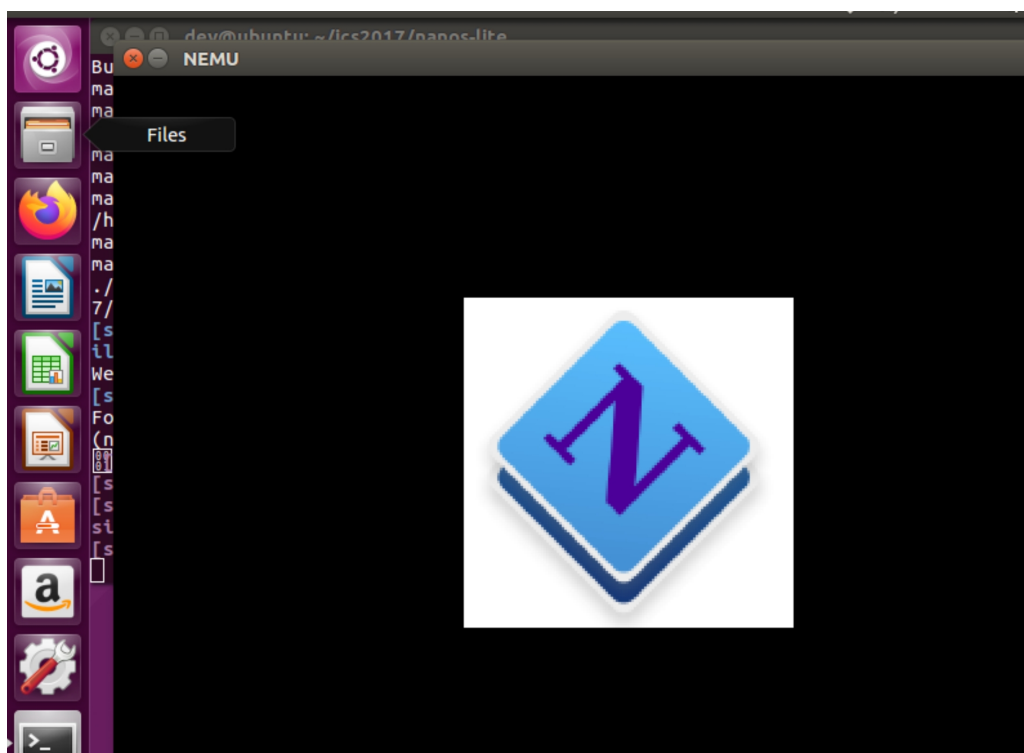


图 3: bmptest 运行结果

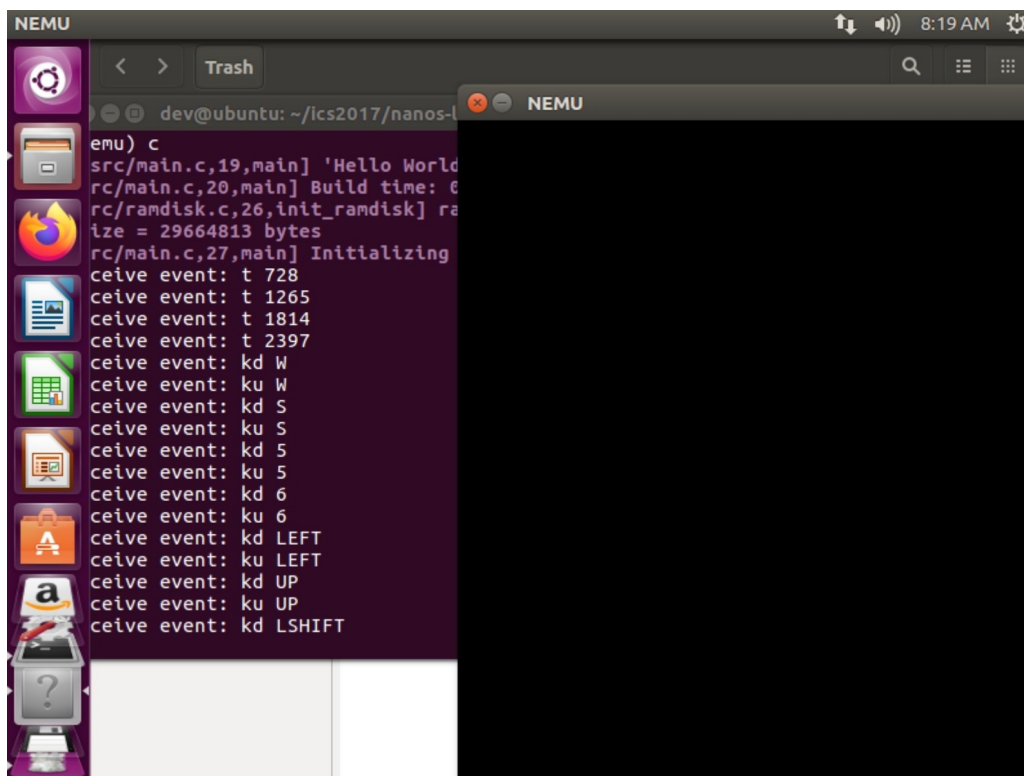


图 4: events 运行结果

iii. 调试问题: bmpstest 黑屏

在实现 VGA 抽象的过程中, 也遇到了一个典型的问题, bmpstest 运行后窗口全黑, Logo 完全没有显示。最初调试时怀疑 `fb_write` 是否漏写了像素、或是否缺少一次刷新, 但其对应的修改都没有效果。为了在改代码前准确定位问题的所在, 在代码中 `fb_write` 与 `dispinfo_read` 内分别插入了 Log 语句来打印每一步运行的状态, 以进行细节的观察。

运行后发现, 终端被 `dispinfo_read` 的日志刷屏, 且 offset 不断递增 (0、1024、2048、...) 却始终读不到结尾, 而 `fb_write` 的日志一条都没有出现。这说明程序根本没有走到绘制环节, 而是陷在了读取 `/proc/dispinfo` 的死循环中。结合代码分析可知, 问题在于 `fs_read` 对 `dispinfo` 这条特殊路径没有做边界裁剪。每次都会返回请求的 1024 字节, 误认为已经读满, 于是按 read 循环读取的 bmpstest 永远认为还没到文件末尾, 从而无限地读下去; 与此同时 `dispinfo_read` 中的 `memcpy` 也早已越过了仅 128 字节的 `dispinfo` 数组边界。

对此, 修复的方法已经很清晰, 就是让 `dispinfo` 也加上普通文件相同的边界裁剪逻辑:

```

1 if (fd == FD_DISPINFO) {
2     if (f->open_offset + len > f->size) len = f->size - f->open_offset;
3     dispinfo_read(buf, f->open_offset, len);

```

```

4   f->open_offset += len;
5   return len;
6 }

```

如此一来，第二次读取时剩余长度为 0，read 返回 0，bmptest 据此判断读取结束并继续绘制，Logo 随即正确显示。当现象不明朗时，先用打印输出确认数据流到底走到了哪一步，往往比直接猜测并修改代码更高效；同时也说明，特殊文件虽然实体不在 ramdisk 中，但其读写在按大小返回、不越界这一逻辑上必须与普通文件保持一致，否则上层依赖返回值判断结尾的逻辑就会失效。

iv. 运行仙剑奇侠传

至此，我们已经通过文件的抽象为《仙剑奇侠传》提供了它所需要的全部能力：读写存档依赖文件系统，更新屏幕依赖 `/dev/fb`，读取按键与查询时钟依赖 `/dev/events`。由于实验框架已经做好了移植，这里我们无需再编写额外的代码，直接将老师提供的 pal 文件放入对应位置即可。

将仙剑的数据文件放入 `navy-apps/fsimg/share/games/pal/` 目录并更新 ramdisk 后，让 Nanos-lite 加载 `/bin/pal`，便可在 NEMU 中运行《仙剑奇侠传》进行游戏。经过测试，运行结果符合预期，PA3 的全部目标至此达成。



图 5: pal 运行结果 - 1

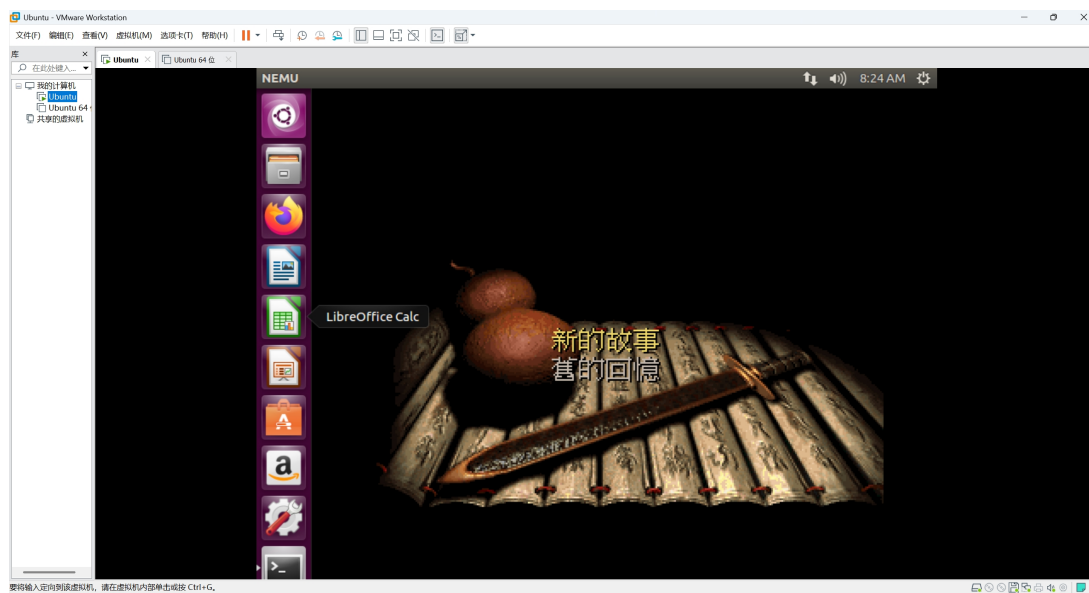


图 6: pal 运行结果 - 2



图 7: pal 运行结果 - 3

二、总结与体会

回顾整个 PA3 的实现过程，我们依然沿用了分阶段、按需推进的开发节奏，并以运行某个具体程序作为每个环节的明确目标。从 dummy 到 hello，再到 text、bmptest、events，直至最终的《仙剑奇侠传》。过程中每一步都建立在前一步的基础之上，又只需验证最新部分的一小段代码，因而当出现报错时，问题的范围总能被限定在最近的改动之内。无论是 dummy 阶段推进时 HIT 到 int、pusha 上这种逐段实现的预期现象，还是 bmptest 黑屏这样意外的 BUG，都能据此快速定位其报错的根源。

与 PA2 中注重于指令如何被执行不同的是，PA3 让我真切地体会到了一条贯穿整个系统的控制流。一次最普通的系统调用，要经由用户程序、库函数、libos、int 指令、NEMU 的中断机制、AM 的现场保存、Nanos-lite 的事件分发与系统调用处理，最后还需要原路返回才能回到用户层面。正是这条看似繁复的链路，配合“一切皆文件”的统一抽象，才使得《仙剑奇侠传》这样颇具规模的程序能够在我们亲手搭建的系统上运行起来。完成 PA3 之后再去回看操作系统，对“系统调用”“异常”“文件”这些概念也不再停留于屏幕上的名词，而是有了从硬件到软件、自底向上的完整理解，让我受益颇丰。

三、必答题

必答题

文件读写的具体过程 仙剑奇侠传中有以下行为：

- ❖ 在 navy-apps/apps/pal/src/global/global.c 的 PAL_LoadGame() 中通过 fread() 读取游戏存档
- ❖ 在 navy-apps/apps/pal/src/hal/hal.c 的 redraw() 中通过 NDL_DrawRect() 更新屏幕

请结合代码解释仙剑奇侠传，库函数，libos, Nanos-lite, AM, NEMU 是如何相互协助，来分别完成游戏存档的读取和屏幕的更新。

在仙剑奇侠传中，PAL_LoadGame 通过 fread 读取游戏存档，redraw 通过 NDL_DrawRect 更新屏幕。两者底层都依赖同一套系统调用与异常机制，差别仅在分发之后的数据去向，下面分别说明。

在游戏存档的读取这部分，可以看作一条读取普通文件的链路。仙剑调用 C 标准库的 fread，它内部转为 Newlib 的底层接口 _read；_read 在 libos 中调用 _syscall_(SYS_read, fd, buf, len)，将参数依次放入 %eax/%ebx/%ecx/%edx 并执行 int \$0x80 陷入内核。NEMU 执行 int 时调用 raise_intr，压栈 EFLAGS/C-S/EIP 后用中断号索引 IDT 跳转到入口；AM 的 vecsys 经 pusha 保存现场、构造陷阱帧并调用 irq_handle，后者封装出 _EVENT_SYSCALL 事件。Nanos-lite 的 do_event 将其转交 do_syscall，分发到 fs_read，由它依据 disk_offset + open_offset 调用 ramdisk_read 把数据读入用户 buf。返回值写回陷阱帧的 %eax，再经 popa/iret 恢复现场，最终层层上交给 fread。整条链路为：fread → _read → _syscall_ → int \$0x80 → raise_intr → vecsys → do_event → do_syscall → fs_read → ramdisk_read，再原路返回。

屏幕更新则对应的是写入特殊文件。前半段与存档读取完全相同。仙剑调用 `NDL_DrawRect`，NDL 库内部用 `lseek` 定位后以 `write` 写入 `/dev/fb`；经 `libos` 的 `_write` 调用 `_syscall(SYS_write, ...)`，同样 `int $0x80` 陷入，并经 `raise_intr`、`vecsyz`、`do_event` 到达 `do_syscall`，分发到 `fs_write`。关键差异在于：`fs_write` 发现 `fd == FD_FB`，即写的是数据不在 `ramdisk` 中的特殊文件，于是重定向到 `fb_write` 而非 `ramdisk_write`。`fb_write` 据字节偏移算出屏幕坐标后调用 AM 的 IOE 接口 `_draw_rect`，将像素写入 NEMU 模拟的 VGA 显存并渲染到窗口。整条链路为：`NDL_DrawRect` → `write` → `_write` → `_syscall_` → `int $0x80` → `raise_intr` → `vecsyz` → `do_event` → `do_syscall` → `fs_write` → `fb_write` → `_draw_rect` → 显存渲染。

总体来看，无论是读存档还是画屏幕，上层都只是在进行“读写文件”这一操作，底层统一通过 `int $0x80` 陷入、查 IDT 跳转、保存现场、分发处理、恢复现场返回这一套机制完成；真正的差别只在 `do_syscall` 分发之后，文件系统依据文件描述符判断目标是普通文件还是设备文件，决定数据流向 `ramdisk` 还是 IOE 设备。这也正是“一切皆文件”抽象的价值，通过统一接口隔离底层设备的差异，使仙剑奇侠传得以在我们实现的简易系统上运行。